Lesson notes · for you

Plans and memory

AI agents · University · Lesson 1

Time
20 min

Materials
Ten sticky notes or small squares of paper; A pen; Your phone, for the lab (Darsi Tiny downloads once, 2.8 MB)

Learning goals

- Compare planning first with planning step by step.
- Explain why errors carry forward, and where to check.
- Budget a context window and choose what stays.

Activity: The ten-note window

1. Set-up (3 min): Work in pairs. One of you is the agent and has only ten sticky notes: that is the whole context window. The other is the person asking.
2. Plan (6 min): The person plans a made-up three-day trip with the agent, one message per turn. The agent writes each turn (question or answer) on a new note and lines them up in order.
3. Overflow (6 min): When the ten notes run out, the agent must choose: throw away the oldest note, or replace three old notes with one short summary note. Try one choice, then the other.
4. Test (5 min): The person asks about day one (the date, the place, the bus time). The agent may answer only from the notes still on the table. Compare which choice kept the facts that mattered.

Any made-up trip will do. If you have no sticky notes, use ten small squares of paper and a desk.

Discussion

- When is planning first better than step by step, and when is it worse?
- Why can’t a model tell you what it has forgotten?
- What would you pin at the top of an agent’s window for your own studies?

Check yourself

- One check an agent needs after every step, and one fact you would pin in its window.

Try it this week
Write the two-line note you would pin at the top of an agent’s window for a week of group study (who, what, when, where), and decide what it may drop.

Worksheet

Plans and memory

AI agents · University · Lesson 1

Name _____ Class _____ Date _____

1 **An agent helps plan a three-day field trip over a long chat. Which memory strategy is safest?**

- ◯ Drop the oldest turns as it fills
- ◯ Keep notes, with the dates and places pinned
- ◯ Keep everything until it's cut off

2 **A conversation is a list of turns, oldest first; each turn is a pair (text, tokens). Write trim(turns, budget): keep the newest turns whose tokens fit in budget, in their original order, and stop at the first turn that doesn't fit. (No model runs in this sandbox, so the token counts are given.)**

```
def trim(turns, budget):
    kept = []
    used = 0
    # walk back from the newest turn
    return kept
```

Write your program:

3 **Match each memory strategy to what happens in the memory lab, or what it is for.**

Keep everything	• • • •	All seven messages work
Drop the oldest	• • • •	The window fills up, then nothing more fits
Keep notes	• • • •	Fetch only the part a step needs
Retrieval	• • • •	"Yes" loses the question it answers

4 **An agent's steps are each right 90% of the time. What does the last line print?**

```
p = 0.9
for steps in [1, 2, 5]:
    print(steps, round(p ** steps, 2))
```

- ◯ 5 4.5
- ◯ 5 0.59
- ◯ 5 0.9

5 **Darsi Tiny's window is 384 tokens, and the app keeps 64 free for the model to write. Fill the gaps: the turns fit if their tokens plus the room to write are at most the window.**

```
def fits(turns, window, reserve):
    used = 0
    for t in turns:
        used += t[1]
    return used + _____ <= _____
```

Word bank: `window` `len(turns)` `reserve` `budget`

✎ **Think of a long task you might hand an agent, such as planning a group project over a week. What must it never forget, and how would you make sure that stays in its window?**

Check your answers

Plans and memory

AI agents · University · Lesson 1

1 **Keep notes, with the dates and places pinned**

Notes keep what was asked and answered in fewer tokens, and pinning the dates and places means the facts the plan depends on never fall out.

2

```
def trim(turns, budget):
    kept = []
    used = 0
    for t in reversed(turns):
        if used + t[1] > budget:
            break
        kept.insert(0, t)
        used += t[1]
    return kept
```

Walking back from the newest turn keeps what the model needs most, and stopping at the first turn that doesn't fit keeps the conversation without gaps.

3 **Keep everything — The window fills up, then nothing more fits**

Drop the oldest — "Yes" loses the question it answers

Keep notes — All seven messages work

Retrieval — Fetch only the part a step needs

Keeping everything fails once the window is full; dropping the oldest loses whatever a later turn depends on; notes keep what was asked and answered in fewer tokens; retrieval keeps long documents outside the window.

4 **5 0.59**

The chance that every step is right multiplies: 0.9 × 0.9 × 0.9 × 0.9 × 0.9 ≈ 0.59.

5

```
def fits(turns, window, reserve):
    used = 0
    for t in turns:
        used += t[1]
    return used + reserve <= window
```

The model needs room to write its next step, so the turns may use only window − reserve tokens: 384 − 64 = 320 for Darsi Tiny's conversation and tools line.